CSC317 Computer Graphics Assignment 4, Part 1 - Meshes

October 15, 2026

Assignment 4: Meshes and Shaders

- Due Date: Friday Nov 13 @ 11:59PM
- Part 1 – Meshes (57/100 marks)
- Part 2 – Shaders (43/100 marks)

Task 1: write_obj.py

- Guide to .obj file format
- For every row of V and F, write one line of the file, starting with what the row represents (“v” for a vertex, “f” for a face)
- Index starts at 1 in .obj but at 0 in Python: add 1 to every face index
- Handle both triangle (m, 3) and quad (m, 4) faces, then return True

```
def write_obj(filename, V, F):
    # filename: output path (str)
    # V: (n, 3) float array
    # F: (m, 3) or (m, 4) int array
    # returns True on success
```

Task 1: write_obj.py

- V: vertex – `v x y z`
- F: face – `f i j k` (triangle) or `f i j k l` (quad)
  - Keep each row's vertex order: faces list their vertices in **counterclockwise** order

```
V = np.array([[0., 0., 0.],
              [1., 0., 0.],
              [1., 1., 0.],
              [0., 1., 0.]])
F = np.array([[0, 1, 2, 3]])
write_obj("square.obj", V, F)
```

```
# square.obj
v 0.0 0.0 0.0
v 1.0 0.0 0.0
v 1.0 1.0 0.0
v 0.0 1.0 0.0
f 1 2 3 4
```

Task 2: cube.py

- Construct the quad mesh of the unit cube $[0, 1]^3$
- Return V (8, 3) float array and F (6, 4) int array
  - In implementation it is just two hard-coded arrays

Task 2: cube.py

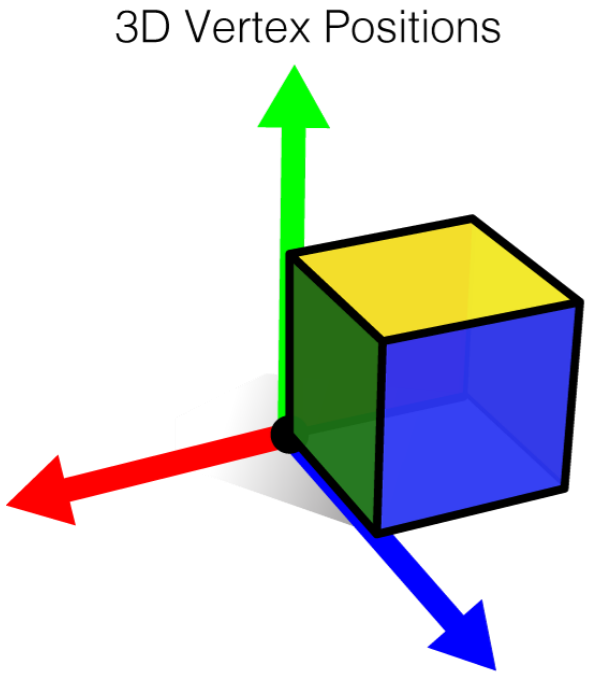
Example:

- V[4] = (0, 0, 1)
- F[0] = (4, 5, 6, 7) → the top face (z = 1)

Each quad is counterclockwise seen from outside, so its normal points out of the cube

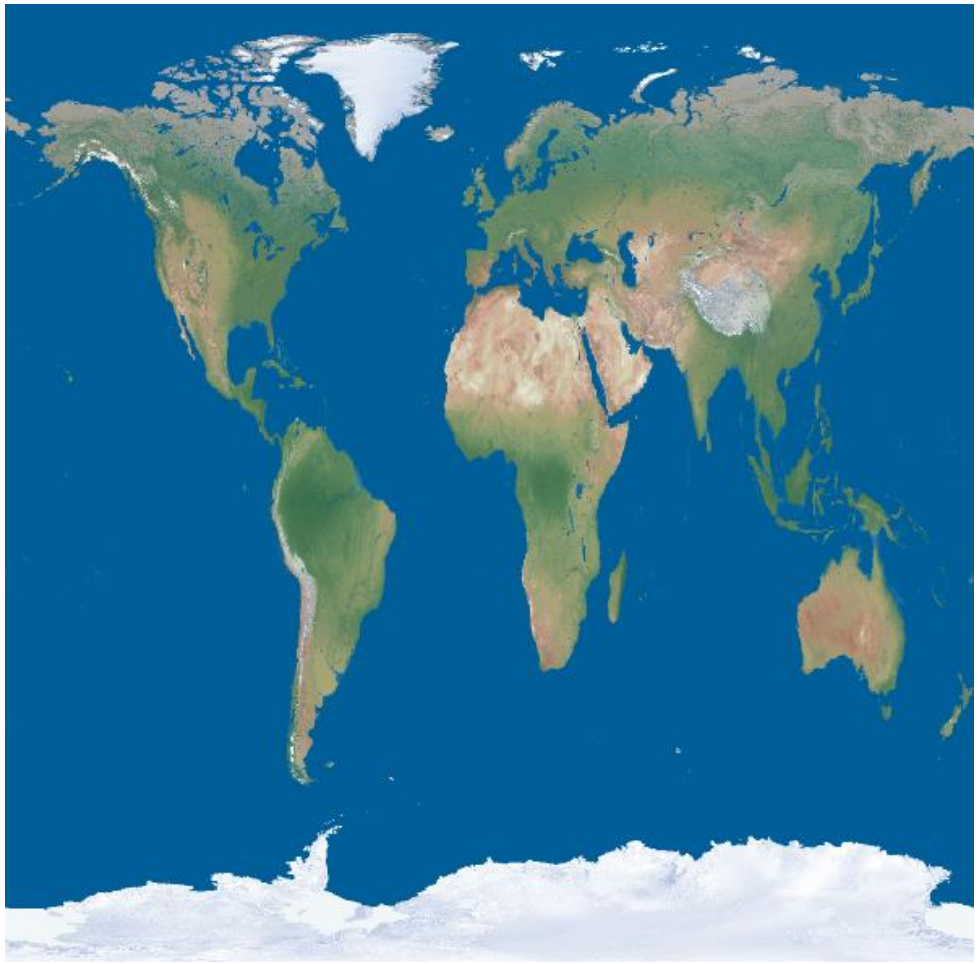
Yours might be in a different order. The tests only check that:

- all coordinates are 0 or 1 and all 8 vertices are used
- each face has 4 distinct corners
- each face's normal points outward



Task 3: sphere.py

- `sphere(num_u, num_v)`: quad mesh of the unit sphere with num_u × num_v faces → think of longitude and latitude of a globe
  - num_u faces around the equator (longitude)
  - num_v faces from pole to pole (latitude)



Task 3: sphere.py

- Map from 2D coordinates (UV) to 3D coordinates (V) using spherical coordinates

$$(u, v) \rightarrow (r, \theta, \phi)$$

- Normal of a unit sphere is the same as its position in spherical coordinates
- Collapse the top and bottom rings to single pole vertices
  - V[0] is the bottom pole (0, −1, 0); V[-1] is the top pole (0, 1, 0)

Task 4: triangle_area_normal.py

- Input: corners a, b, c of a triangle, each a (3,) array
- Output: (3,) normal of the triangle, where the length of the normal is the area of the triangle (not normalized normal)

n: normalized normal, A: area of the triangle, (p0, p1, p2) = (a, b, c)

$$(p_1 - p_0) \times (p_2 - p_0) = 2A \cdot \mathbf{n}$$

Task 5: per_face_normals.py

- Compute per-face unit normals for a triangle mesh
  - Loop over the faces and call triangle_area_normal on each face's 3 corners
  - Then normalize it to unit length (`np.linalg.norm`)

```
def per_face_normals(V, F):
    # V: (n, 3) float, F: (m, 3) int
    # returns N: (m, 3),
    #   N[f] = unit normal of face f
```

Task 6: per_vertex_normals.py

- Compute per-vertex unit normals: `per_vertex_normals(V, F)` → (n, 3)
- Area-weighted average of the normals of the faces around each vertex:

$$\mathbf{n}_v = \frac{\sum_{f \in Nb(v)} A_f \cdot \mathbf{n}_f}{|| \sum_{f \in Nb(v)} A_f \cdot \mathbf{n}_f ||}$$

where A_f is the area of the face (triangle) f, $\mathbf{n}_f$ is the normalized normal of the face (triangle), $Nb(v)$ is the set of faces that contains vertex v.

- Loop over the faces: add each face's triangle_area_normal to each of its 3 vertices, then normalize each vertex's sum

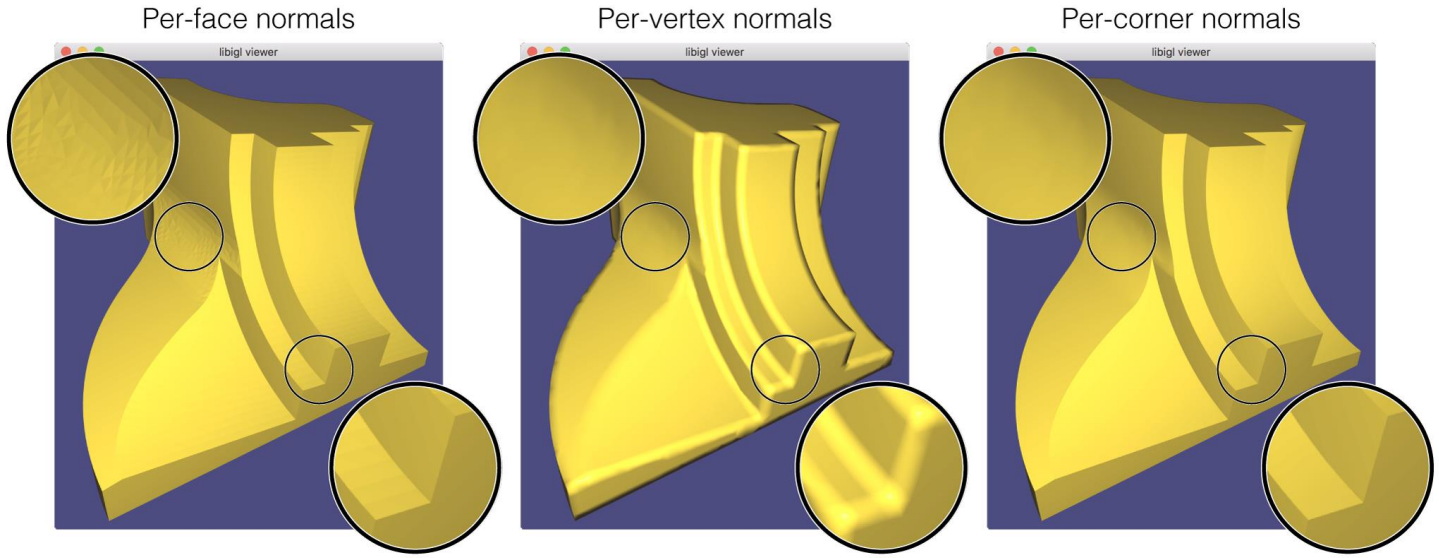
Task 7: vertex_triangle_adjacency.py

- Compute a vertex-triangle adjacency list VF
  - F (num_face, 3) stores the vertex indices of each face
  - VF[v] holds the faces incident on vertex v, in the order the faces are visited ([] if none)
  - You will use it in per_corner_normals (Task 8)

```
def vertex_triangle_adjacency(F, num_vertices):
    # F: (m, k) int array of faces
    # returns VF: list of num_vertices lists,
    #   VF[v] = indices of faces touching v
```

Task 8: per_corner_normals.py

- Per face normal is defined over singular triangles so global smoothness is not guaranteed
- Per vertex normals smooth out sharp edges too much



Task 8: per_corner_normals.py

- Define a normal for every triangle corner (as opposed to each mesh vertex). If the angle between two neighbouring faces is greater than a threshold, do not average the normal

$$\mathbf{n}_{f,c} = \frac{\sum_{g \in Nb(c) | \mathbf{n}_f \cdot \mathbf{n}_g > \epsilon} A_g \cdot \mathbf{n}_g}{|| \sum_{g \in Nb(c) | \mathbf{n}_f \cdot \mathbf{n}_g > \epsilon} A_g \cdot \mathbf{n}_g ||}$$

where A_g is the area of the face (triangle) g, $\mathbf{n}_g$ is the normalized normal of the face (triangle), $Nb(c)$ is the set of faces that contains triangle f corner c.

- Only consider pairs of faces where the dot product of their unit normals is greater than ϵ = cos(threshold)

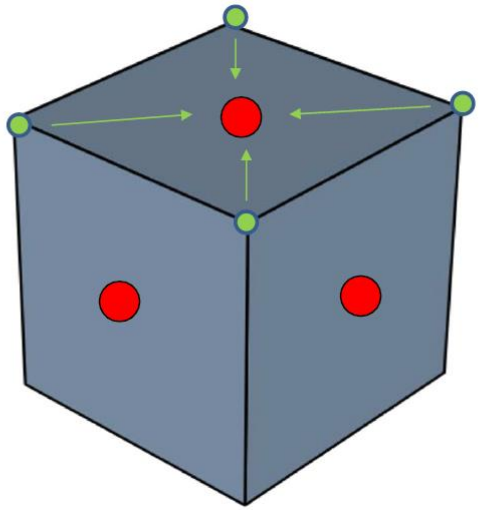
Task 8: per_corner_normals.py

- Use vertex_triangle_adjacency to find the faces around each corner's vertex
- Compare unit-normal dot products with cos(threshold); convert degrees to radians first (np.radians)
- Hint: can use 3 nested for loops

```
def per_corner_normals(V, F,
                       threshold_degrees):
    # V: (n, 3), F: (m, 3)
    # returns N: (3*m, 3) unit normals,
    #   row 3*f + c = corner c of face f
```

Task 9: catmull_clark.py

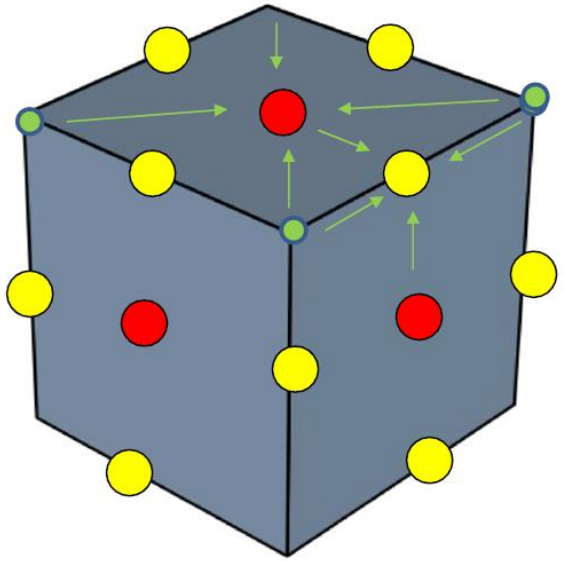
- Step 1: Set the face point for each facet to be the average of its vertices

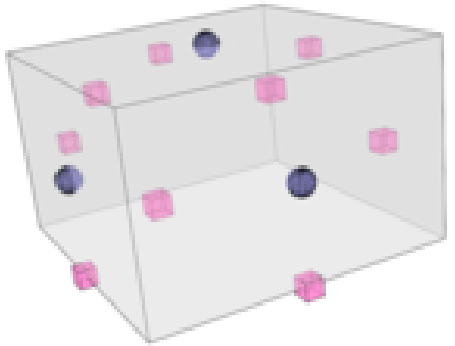


Task 9: catmull_clark.py

- Step 2: Add edge points – average of the two neighbouring face points and edge end points

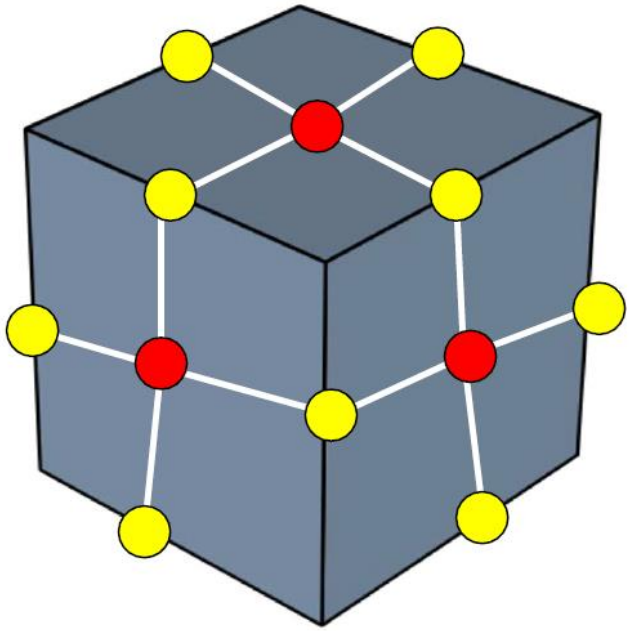
*new edge points are not necessarily on the original edge





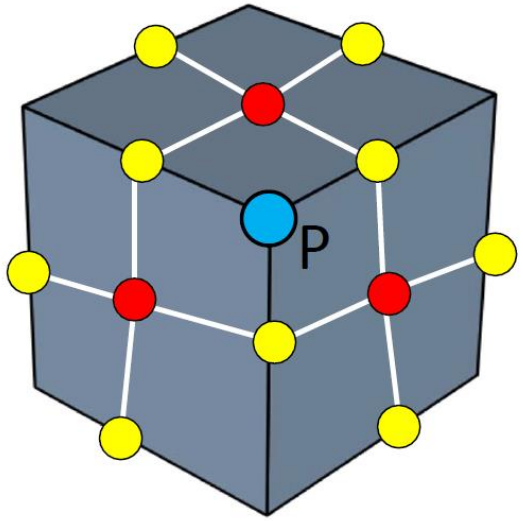
Task 9: catmull_clark.py

- Step 3: Add edges between face points and edge points



Task 9: catmull_clark.py

- Step 4: Move each original vertex according to new position given by:



$$\frac{F + 2R + (n-3)P}{n}$$

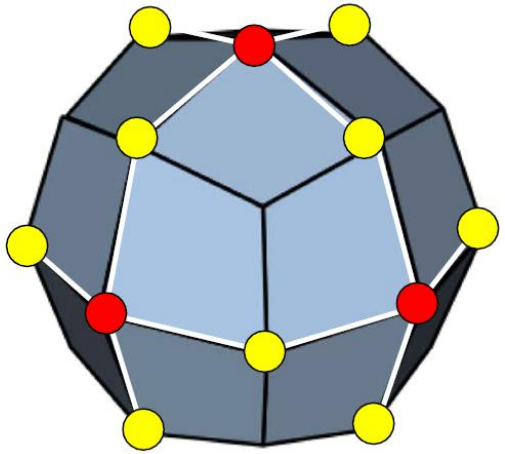
F : Average of all n created face points adjacent to P

R : Average of all original edge midpoints touching P

n = number of edges connected to P

Task 9: catmull_clark.py

- Step 4: Move each original vertex according to new position given by:



$$\frac{F + 2R + (n-3)P}{n}$$

F : Average of all n created face points adjacent to P

R : Average of all original edge midpoints touching P

Task 9: catmull_clark.py

- Step 5: Connect up original points to make facets

